

Contents

SEAS-MFEM v2 Plan: Three Applications + Cross-Phase Wiring	2
1. What v2 is and is not	2
1.1 Goals	2
1.2 Non-goals	2
1.3 Relationship to v1's review findings	2
2. Architecture at a glance	3
3. Cross-phase library libseas_core	3
3.1 Friction (only one law: regularized R&S + aging)	3
3.2 Fault frame (canonical: tangent1 = dip, tangent2 = strike)	4
3.3 Mesh boundary attribute contract	5
3.4 Sign conventions (canonicalized)	6
3.5 Parameter object: one RateStateParams struct	6
4. Application 1: seas_qd (quasi-dynamic only)	7
4.1 Scope	7
4.2 Driver entry point	7
4.3 Reused blocks	8
4.4 What the user sees change	8
5. Application 2: seas_dr (dynamic-rupture only)	8
5.1 Scope	8
5.2 Unified workflow with case-by-case templates	8
5.3 Nucleation strategy contract (addresses v1 R-007)	9
5.4 PML (addresses user §7)	9
6. Application 3: seas_fd (fully dynamic = QD ↔ DR cycle)	10
6.1 Single mesh, single partition (per user §1)	10
6.2 The Hybrid operator	10
6.3 State bridge (DG ↔ DG, addresses v1 R-002, R-005, R-006, and user §6, §10)	12
6.4 Trigger logic and hysteresis (addresses v1 R-010)	13
6.5 Equilibrium-correction sweep (addresses user §8)	13
6.6 Mesh strategy (addresses user §9)	14
6.7 What CAN be PML+loading combined? (addresses user §7)	14
7. TOML configuration (per user §11)	14
7.1 Three schemas, one shared base	14
7.2 Common base (seas_common.toml example)	14
7.3 FD-specific block	15
7.4 CLI shim	15
7.5 Validation	16
8. Workflow per phase (high-level)	16
9. Acceptance criteria (top-level)	16
10. The implementation appendix begins	17
11. File and module layout (full)	17
12. Function-level signatures and unit tests (the long list)	18
12.1 core/friction/rate_state.hpp	19
12.2 core/fault/fault_basis.hpp	19
12.3 core/mesh/boundary_tags.hpp	19
12.4 core/config/toml_loader.hpp	19
12.5 fd/state_bridge.hpp	20
12.6 fd/trigger.hpp	20
12.7 fd/equilibrium_correction.hpp	20
12.8 fd/hybrid_operator.hpp	20
13. Phase plan (the one-page version of §8 expanded)	20
13.1 Phase 0 — Foundations (1 week)	20
13.2 Phase 1 — seas_qd standalone (1 week)	21

13.3 Phase 2 — seas_dr standalone (2 weeks)	21
13.4 Phase 3–6 — seas_fd (4–5 weeks)	21
13.5 Phase Z — Cleanup (0.5 week)	21
13.6 v1 review-finding traceability	21
14. Open questions for v3	22

SEAS-MFEM v2 Plan: Three Applications + Cross-Phase Wiring

Date: 2026-04-29 Status: v2 main draft. Supersedes v1 (unified_framework_and_cycle_wiring_plan_v1.md) including the Round-1 Adversarial Review (§13) embedded in v1. Read order: §1–§9 (main ideas, ~13 pages). §10–§13 (implementation appendix: file lists, signatures, unit tests, phase plan). The body cross-references the appendix by tag (e.g., [A.unit-tests], [A.toml-schema]); skim front-to-back, drill into the appendix only when you want a specific surface pinned down.

1. What v2 is and is not

1.1 Goals

1. Three independent applications. One binary per mode. They share libraries but no driver/runtime path.
 - seas_qd — quasi-dynamic only (long-aseismic, the existing BP5/BP1/BP2 territory).
 - seas_dr — dynamic-rupture only (the existing TPV102/TPV104/TPV205 territory).
 - seas_fd — fully dynamic (= QD ↔ DR cycle): a single run alternates regimes on the same mesh, same MPI partition, same per-rank parameter blocks, no re-initialization.
2. Standalone, non-interfering branches. Each application has its own driver, its own config, its own test suite. A bug in seas_dr cannot reach seas_qd. The “do not edit” zone from v1 is dropped — the user has authorized any structural change — but the working benchmarks (BP5 v62, TPV102/104/205) must keep reproducing their goldens through every phase.
3. Reusable cross-phase library. A shared core (friction, fault-frame, mesh boundary contract, state bridge) eliminates today’s sign/parameter discrepancies between QD and dynamic.
4. Single-mesh single-partition for seas_fd. Each MPI rank holds the QD and dynamic state for its own subdomain; switching regimes flips a flag and rewires which operator advances time, no halo exchange beyond what each phase already does.
5. TOML-first configuration. Designed in this plan, implemented in Phase 0; CLI flags become a thin shim over TOML. Three TOML schemas (one per mode), one shared sub-schema for cross-cutting items (mesh, friction, output).

1.2 Non-goals

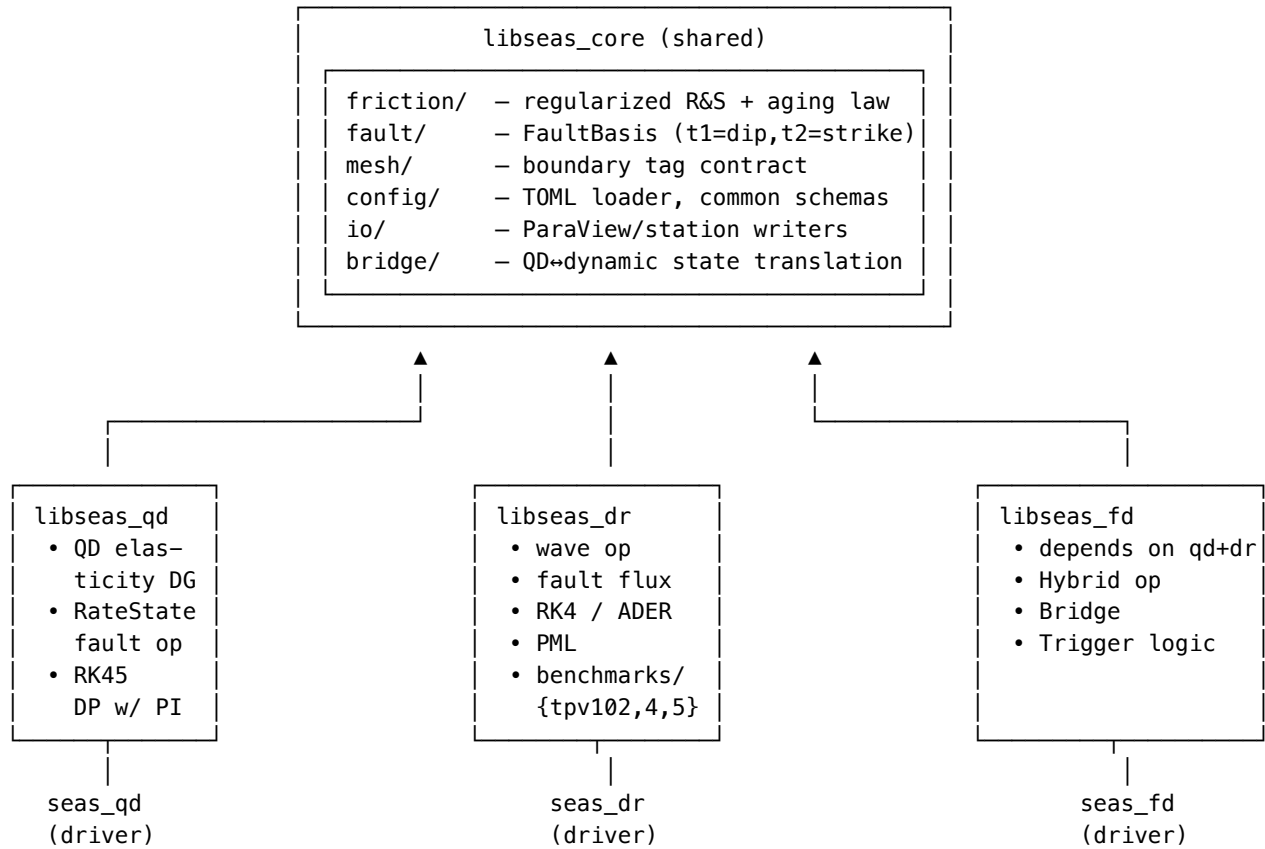
- No LSW (linear slip-weakening) anywhere in seas_qd or seas_fd. Per user direction: only regularized rate-and-state friction with the aging law and its invariants are supported by seas_qd and seas_fd. TPV205 (LSW) keeps living inside seas_dr only; that is the seas_dr mode’s prerogative.
- No coordinate-based boundary detection (today’s FindFaultSharedFaces falls back to $|Y| < \text{coord_tol}$ for shared faces — gone in v2). Every face’s role comes from a Gmsh Physical tag.
- No second mesh partition in seas_fd. Re-partitioning at handoff is forbidden.
- No CG↔DG projection. v1 invoked one; both phases are DG (verified: domain/elasticity_operator.hpp:439 uses DG_FECollection; dynamic/wave_operator.hpp:608 uses L2_FECollection). State bridging is DG↔DG (§5.3).

1.3 Relationship to v1’s review findings

Every CRITICAL/MODERATE finding from v1 §13 is addressed in this plan (cross-reference table in §13.6). The two largest changes from v1: - v1’s “unified driver” (one binary, multiple modes) is replaced by v2’s three

separate drivers (per user §4 of the suggestions). - v1's CG→DG projection (§4.4 step 3) is replaced by a DG↔DG identity for the bulk and a fault-DOF copy for fault state (§5.3).

2. Architecture at a glance



Three libraries (qd, dr, fd) on top of one core. Three drivers, one per app. Adding a new app is a new library + a new driver, not a new mode in an existing driver.

3. Cross-phase library `libseas_core`

These are the surfaces that today exist in subtly different forms in QD and dynamic and are the primary source of cross-phase bugs. v2 unifies them.

3.1 Friction (only one law: regularized R&S + aging)

Decision: `seas_qd` and `seas_fd` use exactly one friction law. `seas_dr` keeps its three (R&S, R&S+SRW, LSW) for backward compat with TPV benchmarks but their implementations call the unified surface where they overlap.

The unified surface is one stateless function and one stateless ODE evaluator:

```
// core/friction/rate_state.hpp
namespace seas::friction {
```

```

/// Regularized rate-and-state friction coefficient.
///  $f(V, \psi, a) = a \cdot \operatorname{asinh}[ (V / 2V_0) \cdot \exp(\psi / a) ]$ 
/// Used identically by QD's RateStateFault and by dynamic's FaultFaceFlux.
inline real_t FrictionCoefficient(real_t V_abs, real_t psi, real_t a, real_t V0);

/// Solve the friction balance  $\sigma_n \cdot f(V, \psi, a) + \eta \cdot V = \tau_{\text{total}}$  for  $V_{\text{abs}}$ .
/// Brent in  $\log_{10}(V)$  space. Returns  $V_{\text{abs}} \geq 0$ . Returns frictionless limit
///  $\tau_{\text{total}}/\eta$  when bracket is degenerate (per debug v7 H1).
real_t SolveBalanceForVabs(
    real_t tau_total_abs, real_t sigma_n, real_t psi,
    real_t a, real_t V0, real_t eta_s,
    real_t tol = 1e-12, int max_iter = 200);

/// Aging-law steady advance over dt (analytic update from Tandem):
///  $d\psi/dt = (b \cdot V_0 / Dc) \cdot [\exp((f_0 - \psi) / b) - V_{\text{abs}} / V_0]$ 
/// Integrated analytically over a stage of constant  $V_{\text{abs}}$ .
real_t AdvancePsiAging(
    real_t psi_n, real_t V_abs, real_t dt,
    real_t a, real_t b, real_t Dc, real_t f0, real_t V0);

} // namespace seas::friction

```

One source for **FrictionCoefficient**. Today the regularized formula lives in friction/friction_coeff_stable.hpp (QD path) and is re-derived in dynamic/friction_solver.{hpp,cpp} and dynamic/tpv102_* (dynamic path). v2 collapses these to a single inline function that both sides include. The QD operator's friction call site, the dynamic Brent solver, the LSW-free TPV102/104 paths, and the bridge's verification all call this exact function. This eliminates the v1 R-002 risk of QD and dynamic disagreeing on the friction zero-point at handoff.

One source for the Brent solver. friction/dieterich_ruina.hpp (QD) and dynamic/friction_solver.cpp (dynamic) both implement Brent in $\log_{10}(V)$ with a frictionless-limit fallback. v2 keeps exactly one (core/friction/rate_state.hpp::SolveBalanceForVabs) — the QD operator and the dynamic FaultFace-Flux both call it.

Aging-law analytic advance is shared. Today QD uses friction/state_evolution.hpp::UpdateStateAnalytic; dynamic uses an inlined version inside the per-substep iterators. v2 promotes the QD version to core/friction/rate_state.hpp::AdvancePsiAging and deletes the dynamic copies.

3.2 Fault frame (canonical: tangent1 = dip, tangent2 = strike)

Per miniapps/seas/CLAUDE.md: the project-wide convention is tangent1 = dip, tangent2 = strike. Today, two different builders coexist: - fault/fault_basis.hpp (BP5): t1 = dip, t2 = strike (project-wide convention). - dynamic/godunov_flux.hpp::BuildFrame (used pre-R-801 by interior fault branch): t1 = strike, t2 = dip.

R-801 already unified them onto BP5. v2 makes that permanent by deleting **BuildFrame** and routing every dynamic-side caller through core/fault/fault_basis.hpp::FaultBasis — same code, same signature, no choice.

```

// core/fault/fault_basis.hpp (promoted from miniapps/seas/fault/fault_basis.hpp)
namespace seas::fault {

class FaultBasis {
public:
    /// Build the fault-local orthonormal frame from a face normal + project-wide up.
    /// Output: n (normal), t1 (dip, points downward into earth), t2 (strike).

```

```

FaultBasis(const Vector &face_normal, const Vector &world_up = {0, 0, 1});

const Vector &Normal() const { return n_; }
const Vector &Tangent1() const { return t1_; } // dip, (0,0,-1) for vertical y=0 fault
const Vector &Tangent2() const { return t2_; } // strike, (+1,0,0) for vertical y=0 fault

/// Rotate a 3-vector v_world into fault-local (n, t1, t2).
void RotateWorldToFault(const real_t v_world[3], real_t v_fault[3]) const;
void RotateFaultToWorld(const real_t v_fault[3], real_t v_world[3]) const;

/// Rotate a symmetric 3x3 stress tensor (Voigt) world→fault.
void RotateStressWorldToFault(const real_t s_world[6], real_t s_fault[6]) const;
void RotateStressFaultToWorld(const real_t s_fault[6], real_t s_world[6]) const;
};

} // namespace seas::fault

```

Single contract: tangent1 = dip = (0, 0, -1) for a vertical y=0 fault, tangent2 = strike = (+1, 0, 0). All D0FData::tau1_* slots are dip components; D0FData::tau2_* are strike; slip1, v1 likewise. This is the existing convention (debug v7.0.0 R-801) — v2 freezes it and removes the alternative.

3.3 Mesh boundary attribute contract

Rule: every face's role is determined by its Gmsh Physical tag. No coordinate-based fallback.

Today domain/seas_boundary_tags.hpp::FaultBoundaryData::FindFaultSharedFaces falls back to $|Y| < \text{coord_tol}$ because MFEM doesn't expose boundary attributes on shared faces directly. v2 fixes the underlying problem instead of papering over it.

Contract:

```

// core/mesh/boundary_tags.hpp
namespace seas::mesh {

/// SCEC-SEAS standard tags (Gmsh Physical Surface IDs).
struct BoundaryTags {
    static constexpr int FAULT      = 3; // The interior fault surface
    static constexpr int NATURAL    = 1; // Top + bottom (zero traction)
    static constexpr int DIRICHLET  = 5; // Vertical far-field faces (plate loading)
    static constexpr int PML       = 7; // Outer PML (only used by dr/fd)
};

/// Build a per-rank shared-face-attribute table at mesh-load time.
/// At partition time we know each shared face's *interior face index* on the
/// owner rank. Owner rank publishes the boundary attribute it would have
/// assigned (e.g. FAULT) by writing it into a small ParGridFunction / Array,
/// then ParMesh::ExchangeFaceNbrData collectives propagate to non-owner.
/// Result: every rank knows the fault tag of every shared face it touches,
/// without any coordinate lookup.
class SharedFaceTagTable {
public:
    void Build(ParMesh &pmesh);
    int GetSharedFaceTag(int shared_face_idx) const;
    bool IsFault(int shared_face_idx) const { return GetSharedFaceTag(shared_face_idx) == BoundaryTags::FAULT; }
};

```

```
} // namespace seas::mesh
```

Implementation note (more in §10): because MFEM 4.x exposes `bdr_attributes` only on locally-owned boundary elements, the build pass uses an `Array<int>` packed by face index, summed over MPI_MIN_LOC-style reductions on shared faces — i.e., the owner's tag wins. This gives a tag-based answer for every face, including shared ones, with no coordinate detection.

Mesh-side requirement. All meshes used by `seas_qd`, `seas_dr`, `seas_fd` must tag with these IDs. The TPV102 inline mesh generator (currently in each TPV driver) must emit the same tags. The existing BP5 mesh `bp5/mesh/bp5_*.msh` already conforms.

3.4 Sign conventions (canonicalized)

Three places in the current code disagreed at one time or another (per debug docs v8 / v10 / v13). v2 freezes them in `core/conventions.hpp` as compile-time constants and a small contract:

Quantity	Sign	Source of truth
Slip rate vector direction	parallel to traction τ_{vec}	<code>core/friction/rate_state.hpp::DecomposeV</code> (debug v8)
Pre-stress vector direction	parallel to initial velocity V_{init}	TOML field <code>pre_stress.parallel_to = "V_init"</code>
Dip direction	(0, 0, -1), downward into earth	<code>core/fault/fault_basis.hpp</code> (debug v10)
Normal stress	$\sigma_n > 0$ = compression	<code>core/conventions.hpp::SIGMA_N_COMPRESSION_P</code>
Depth coord	$z = 0$ at surface, $z < 0$ is depth	<code>core/conventions.hpp::DEPTH_NEGATIVE</code>
Slip rate threshold for QD→DR	$V_{\text{max}} > 1\text{e-}3$ m/s (BP5 spec)	<code>core/conventions.hpp::V_ACTIVATE_DEFAULT</code>
Slip rate threshold for DR→QD	$V_{\text{max}} < 1\text{e-}4$ m/s (per user; tunable, see §6.4)	<code>core/conventions.hpp::V_DEACTIVATE_DEFAULT</code>

```
// core/conventions.hpp
namespace seas {
constexpr real_t V_ACTIVATE_DEFAULT    = 1e-3; // m/s, QD → DR
constexpr real_t V_DEACTIVATE_DEFAULT = 1e-4; // m/s, DR → QD; user note: must be > Vp ~ 1e-9
constexpr real_t SIGMA_N_COMPRESSION_POSITIVE = 1.0; // sentinel; compile-time guard
}
```

Important: these defaults override v1's `V_deactivate = 1e-6` m/s (which v1's review correctly flagged but with the inequality inverted). Per user direction:

- `V_activate = 1e-3` m/s — well above plate rate $V_p \approx 3.17 \times 10^{-10}$ m/s (BP5: 10^{-9} m/s order; SCEC plate rate 1×10^{-9} m/s).
- `V_deactivate = 1e-4` m/s (default) — three orders of magnitude above plate rate, so the QD steady state has plenty of relaxation runway.
- The two thresholds straddle a roughly two-order-of-magnitude band so the regime hysteresis prevents rapid toggling.
- Both defaults are TOML-overridable (`[trigger] V_activate = 1e-3, V_deactivate = 1e-4`). For BP5 cycle reproduction we will sweep `V_deactivate` $\in \{1\text{e-}4, 1\text{e-}5, 1\text{e-}6\}$ and document the chosen value.

3.5 Parameter object: one `RateStateParams` struct

Today QD passes friction parameters as (a, b, Dc, f0, V0, V_init, V_w?) through `BP5Params` and dynamic passes them through per-DOF `DOFData` fields. v2 has one struct used at config time and one struct used at runtime:

```

// core/friction/rate_state_params.hpp
namespace seas::friction {

// Spatial parameter functions (config-time).
// Same struct used by seas_qd and seas_fd (and by seas_dr's R&S benchmarks).
struct RateStateFields {
    std::function<real_t(real_t x, real_t y, real_t z)> a;    // direct effect
    std::function<real_t(real_t x, real_t y, real_t z)> b;    // evolution effect
    std::function<real_t(real_t x, real_t y, real_t z)> Dc;    // critical slip distance
    real_t f0 = 0.6;
    real_t V0 = 1e-6;    // reference velocity
    real_t V_init = 1e-9;    // initial slip rate (Vp for BP5)
};

// Per-DOF runtime values (filled at init from RateStateFields by
// InitializeFaultDOFs; carried by DOFData for the dynamic side and
// by parallel grid functions for the QD side; identical numerics).
struct RateStatePerDOF {
    real_t a;
    real_t b;
    real_t Dc;
    real_t f0;
    real_t V0;
};

} // namespace seas::friction

```

QD's RateStateFaultOperator is rewritten to read RateStateFields at init and store per-DOF RateStatePerDOF blocks. Dynamic's DOFData already carries a, Dc per QP; v2 also adds b, f0, V0 (which are uniform per benchmark today, but must be per-DOF for cycle-mode generality and zero-cost for benchmarks that use uniform values).

4. Application 1: seas_qd (quasi-dynamic only)

4.1 Scope

Drives the existing BP5 / BP1 / BP2 problems and any future quasi-dynamic R&S problem. Only friction law: regularized R&S + aging. RK45 (Dormand-Prince) with PI controller. MUMPS direct elasticity solve per RHS.

4.2 Driver entry point

```

// drivers/seas_qd_driver.cpp
int main(int argc, char* argv[]) {
    seas::MPIContext mpi(&argc, &argv);
    auto cfg = seas::config::LoadQDConfig(argc, argv);    // TOML, see §7
    auto mesh = seas::mesh::LoadParMesh(cfg.mesh, mpi);    // honors §3.3 contract
    seas::qd::Application app(cfg, mesh);    // owns elasticity + fault op + RK45
    app.Run();    // returns normally on tfinal
    return 0;
}

```

seas::qd::Application is the surface that today is split between solver/seas_operator.hpp, drivers/seas_driver.cpp's 2000-line main loop, and the BP5-specific initialization in bp5/.... v2

promotes the time loop, the 4-phase initialization (Prelim → solve → Init → Verify), the $V_{\max}$ event detection, and the I/O cadence into one class so `seas_fd` can call its `Step()` directly (§6.2).

4.3 Reused blocks

- `core/friction/rate_state.hpp` (entire surface) — new, replaces today's `friction/dieterich_ruina.hpp` and `friction/state_evolution.hpp` (those become tiny shims forwarding to core for one release; deleted in §13.5 phase Z).
- `core/fault/fault_basis.hpp` — new, identical to today's `fault/fault_basis.hpp`.
- `core/mesh/boundary_tags.hpp` — new, replaces `domain/seas_boundary_tags.hpp` (existing class kept only as a typedef for one release).
- Existing `domain/elasticity_operator.hpp` — kept; its DG assembly is correct and well-tested. Internal calls to friction/fault are routed to `core/`.

4.4 What the user sees change

- TOML config replaces ~20 CLI flags (existing flags continue to work via a TOML-shim translator for one release; see §7.4).
- Friction call sites all route through one function (no behavioral change at fixed parameters).
- BP5 v62 golden reproduces byte-for-byte.

5. Application 2: `seas_dr` (dynamic-rupture only)

5.1 Scope

The existing TPV102 / TPV104 / TPV205 territory. Per user: this app keeps all three friction laws (R&S, R&S+SRW, LSW) — they are TPV verification benchmarks. Inside `seas_dr` we still take the v1 framework opportunity to collapse the per-benchmark trio of files into a unified workflow with per-benchmark templates.

5.2 Unified workflow with case-by-case templates

Per user §3 of the suggestions: keep templates per case but unify the workflow. The shape is the v1 strategy-tag pattern (R&S vs LSW friction, aging vs slip-law-SRW vs no-state evolution, overwrite vs accumulate vs static nucleation), but every TPV benchmark goes through the same time-loop scaffold.

```
// dr/framework/runtime_loop.hpp
template <class FrictionTag, class StateTag, class NucleationTag>
class DRRuntimeLoop {
public:
    DRRuntimeLoop(const seas::dr::BenchmarkConfig &cfg, ParMesh &mesh, MPI_Comm comm);
    void Run();    // wave-RK4 + ADER predictor + per-substep iterator + station/I0

private:
    seas::dr::WaveOperator wave_op_;
    seas::dr::FaultFaceFlux fault_flux_;
    std::vector<seas::dr::DOFData> dof_data_;
    seas::dr::SubStepIterator<FrictionTag, StateTag, NucleationTag> iterator_;
    seas::io::StationWriter stations_;
};

using TPV102Loop = DRRuntimeLoop<RateStateNewtonTag, AgingLawPsiTag, OverwriteNucleationTag>;
using TPV104Loop = DRRuntimeLoop<RateStateNewtonTag, SlipLawSRWPsiTag, AccumulateNucleationTag>;
using TPV205Loop = DRRuntimeLoop<LSWClosedFormTag, NoStateTag, StaticNucleationTag>;
```


The driver dispatches by TOML [benchmark] field:

```
// drivers/seas_dr_driver.cpp
int main(int argc, char* argv[]) {
    seas::MPIContext mpi(&argc, &argv);
    auto cfg = seas::config::LoadDRConfig(argc, argv);
    auto mesh = seas::mesh::LoadParMesh(cfg.mesh, mpi);

    if (cfg.benchmark == "tpv102") { TPV102Loop loop(cfg, mesh, MPI_COMM_WORLD); loop.Run(); }
    else if (cfg.benchmark == "tpv104") { TPV104Loop loop(cfg, mesh, MPI_COMM_WORLD); loop.Run(); }
    else if (cfg.benchmark == "tpv205") { TPV205Loop loop(cfg, mesh, MPI_COMM_WORLD); loop.Run(); }
    else { mfem::mfem_error("unknown benchmark"); }
    return 0;
}
```

Adding a fourth TPV benchmark is one new TOML, one new typedef, optionally one new strategy specialization. No interference with `seas_qd` or `seas_fd` — the dispatch lives entirely inside `seas_dr_driver.cpp`.

5.3 Nucleation strategy contract (addresses v1 R-007)

```
// dr/framework/nucleation_strategy.hpp
namespace seas::dr {

    /// Where the strategy writes its perturbation. Must be fixed at compile time
    /// per-tag so the iterator and the friction solver stay in agreement.
    enum class NucleationSlot { TauNuc, Tau0, NoOp };

    struct OverwriteNucleationTag    { static constexpr NucleationSlot slot = NucleationSlot::Tau0; }; // TPV
    struct AccumulateNucleationTag   { static constexpr NucleationSlot slot = NucleationSlot::TauNuc; }; // TPV
    struct StaticNucleationTag       { static constexpr NucleationSlot slot = NucleationSlot::NoOp; }; // TPV

    template <class Tag>
    struct NucleationStrategy {
        /// Called once per sub-step. The implementation:
        /// - Overwrite tag: writes the smoothStep value to tau2_0 only when sub_step_index == 0
        ///   (then no-op for sub_step_index > 0 within the same macro step).
        /// - Accumulate tag: writes the per-substep increment to tau2_nuc on every call;
        ///   iterator clears tau2_nuc at the start of every macro step.
        /// - Static tag: no-op.
        static void Apply(std::vector<DOFData> &dof_data,
            const Vector &fault_x, const Vector &fault_z,
            const NucleationParams &params,
            real_t t_macro, real_t dt_sub, int sub_step_index);
    };
}
```

5.4 PML (addresses user §7)

`dynamic/pml_layer.{hpp,cpp}` (Komatitsch-Martin CPML, cubic ramp) is the existing PML. v2 keeps and finishes verifying it (today: PML works for plane-wave incidence; not yet verified for wide-angle / surface-wave incidence — see §13 for the verification plan).

Coexistence with boundary loading? Yes, with a clear partition. The mesh has separate Physical Surface tags (per §3.3): tag 5 = Dirichlet (plate-loading vertical faces), tag 7 = PML outer (absorbing). These are

non-overlapping faces.

seas_qd: PML is unused. The wave-propagation modes that PML absorbs do not exist in the quasi-static limit. Loading is via Dirichlet on tag-5 faces.

seas_dr: PML is on tag-7 faces. Loading is also Dirichlet on tag-5 faces — TPV102/104/205 do not actually load (their pre-stress is static), but the mechanism is the same as BP5.

seas_fd: PML always on, Dirichlet loading always on — one mesh, both contracts active. The two are spatially disjoint (tag-5 vs tag-7 faces), so they coexist trivially. The QD phase ignores PML (no waves to absorb); the DR phase uses both.

For BP5 cycle-mode meshes we add a tag-7 outer layer ~5–10 km thick around the existing tag-5 / tag-1 box; the BP5 production mesh requires regeneration with this added layer. (Verification mesh per §6.5 has the same topology at smaller scale.)

6. Application 3: **seas_fd** (fully dynamic = QD ↔ DR cycle)

This is the new application. Everything below is new code, but heavily reuses §3 core and §4–§5 libraries.

6.1 Single mesh, single partition (per user §1)

seas_fd constructs one **ParMesh** at startup, partitioned once. Each MPI rank holds: - The bulk DG state **Q** (9 components per DG dof) — used by the dynamic phase. - The QD parallel grid functions for displacement and accumulated slip — used by the QD phase. - The fault-DOF state [**s_dip**, **s_strike**, **ψ**] per local fault DOF — shared between phases (same numerical values, same convention; §3.5). - The **D0FData** array per local fault QP — used by the dynamic phase only (initialized at startup AND on QD→DR handoff; otherwise dormant).

No re-partition at handoff. No re-initialization of mesh, partition, MPI communicators, or the static fault-DOF parameter blocks (**a**, **b**, **Dc**, **f0**, **V0**). Only the per-DOF dynamic-state slots in **D0FData** (slip1, slip2, **ψ**, **V1**, **V2**, **tau*_corr**, **tau*_trial**) are written at handoff; the rest are filled once and reused for every cycle.

6.2 The Hybrid operator

```
// fd/hybrid_operator.hpp
namespace seas::fd {

class HybridOperator {
public:
    HybridOperator(const FDConfig &cfg, ParMesh &mesh);

    /// Owns the time loop directly – does NOT inherit TimeDependentOperator
    /// (resolves v1 R-003: state size changes per regime, MFEM ODESolver is wrong fit).
    /// Driver calls Step(t, dt) in a while-loop until t >= tfinal.
    void Step(real_t &t, real_t &dt);

    /// Cycle telemetry.
    int EventCount() const { return event_count_; }
    bool InQuasiDynamic() const { return regime_ == Regime::QD; }

private:
    enum class Regime { QD, Dynamic };
    Regime regime_ = Regime::QD;
};
```

```

seas::qd::Application qd_app_;    // owns elasticity + fault op + RK45 + 4-phase init
seas::dr::WaveOperator dyn_wave_;
seas::dr::FaultFaceFlux dyn_flux_;
std::vector<seas::dr::DOFData> dof_data_;
seas::dr::SubStepIterator<RateStateNewtonTag, AgingLawPsiTag, NoNucleationTag> iter_;
    // Note: RateStateNewtonTag + AgingLawPsiTag is the only combination supported in seas_fd
    // (per user §5: only R&S aging law). NoNucleationTag is a new no-op tag (cycle mode
    // does not pre-stress nucleate; events grow naturally from QD slip → V_max breach).

seas::fd::StateBridge bridge_;    // §6.3
seas::fd::Trigger trigger_;      // §6.4

real_t V_activate_;
real_t V_deactivate_;
real_t exit_grace_t_;    // seconds V_max must stay below V_deactivate before regime switch

int event_count_ = 0;
real_t t_below_thresh_ = 0;
};

} // namespace seas::fd

```

Step() advances by one logical macro-step using whichever phase is active, then evaluates the trigger. Pseudocode (full version in §10):

```

void HybridOperator::Step(real_t &t, real_t &dt) {
    if (regime_ == Regime::QD) {
        qd_app_.StepOne(t, dt);    // RK45 step
        real_t Vmax = AllReduceVMax(qd_app_.SlipRate());
        if (Vmax > V_activate_) {
            bridge_.HandoffQDtoDynamic(qd_app_, dof_data_, dyn_wave_); // §6.3
            regime_ = Regime::Dynamic;
            dt = cfg_.dt_dyn_initial;
            ++event_count_;
        }
    } else {
        dyn_wave_.Step(dof_data_, t, dt, iter_);    // RK4 + ADER + iterator
        real_t Vmax = AllReduceVMaxFromDOFData(dof_data_);
        if (Vmax < V_deactivate_) {
            t_below_thresh_ += dt;
            if (t_below_thresh_ > exit_grace_t_) {
                bridge_.HandoffDynamicToQD(dof_data_, dyn_wave_, qd_app_); // §6.3
                regime_ = Regime::QD;
                dt = cfg_.dt_qd_default;
                t_below_thresh_ = 0;
            }
        } else {
            t_below_thresh_ = 0;
        }
    }
}
}

```

6.3 State bridge (DG ↔ DG, addresses v1 R-002, R-005, R-006, and user §6, §10)

There is no CG anywhere. Both phases live on DG / L^2 spaces. The bridge translates between two different DG state representations on the same mesh, same partition:

- QD state per local fault DOF: (s_{dip} , s_{strike} , ψ) carried in a ParGridFunction-like AOS layout in `qd_app_`.
- DR state per local fault QP: `DOFData` block — already includes (s_{lip1} , s_{lip2} , ψ , V_1 , V_2 , τ_{corr} , τ_{trial} , τ_{nuc} , σ_{n0} , $\sigma_{\text{n_corr}}$, a , b , D_c , f_0 , V_0).
- DR bulk state per local DG dof: $Q[9] = (\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{xy}, \sigma_{xz}, \sigma_{yz}, v_x, v_y, v_z)$.

Today (DG↔DG, same mesh) the inter-space map for the fault state is a direct copy plus a basis-rotation (fault DG nodal layout is the same as fault QP layout for nodal DG; no projection needed). For the bulk state, the QD operator does not store a velocity field but does store the displacement field via its own DG basis; v2 makes this explicit by adding `qd_app_.GetDisplacementGridFunction()` returning the parallel DG u .

```
// fd/state_bridge.hpp
namespace seas::fd {

class StateBridge {
public:
    StateBridge(const FDConfig &cfg, ParMesh &mesh);

    /// QD → Dynamic at a triggered event.
    /// 1. Re-solve QD elasticity at exact handoff time  $t$  to get  $u(x)$ .
    /// 2. Compute  $\sigma(x) = C : \varepsilon(u)$  elementwise (DG, no global solve).
    /// 3. Initialize bulk  $Q$ : pack  $\sigma + v$  ( $v$  from §6.3.2) into the wave op's DG state.
    /// 4. Pack DOFData per fault QP from QD fault-DOF state ( $s_{\text{lip}}$ ,  $\psi$ ,  $V$ ).
    /// 5. Run  $N_{\text{settle}}$  (default 10) wave steps with elevated PML damping (§6.5).
    void HandoffQDtoDynamic(seas::qd::Application &qd_app,
                           std::vector<seas::dr::DOFData> &dof_data,
                           seas::dr::WaveOperator &dyn_wave);

    /// Dynamic → QD at a triggered quiet period.
    /// 1. Capture ( $s_{\text{lip1}}$ ,  $s_{\text{lip2}}$ ,  $\psi$ ) per fault QP from DOFData.
    /// 2. Copy fault-QP→fault-DOF (identity for nodal DG with same order).
    /// 3. Verify friction equilibrium with  $V_{\text{qd}}$  back-out (§6.3.3).
    /// 4. Reset QD time stepper ( $dt = \text{cfg}.dt_{\text{qd\_default}}$ ), discard bulk  $Q$ .
    void HandoffDynamicToQD(const std::vector<seas::dr::DOFData> &dof_data,
                           seas::dr::WaveOperator &dyn_wave,
                           seas::qd::Application &qd_app);
};

} // namespace seas::fd
```

6.3.1 Fault-state copy is exact (no projection) Because both phases live on the same parallel mesh with the same DG polynomial order on the fault, the fault DOF layout in QD and the fault QP layout in DR are the same set of points (nodal DG: dofs are QPs at the Gauss-Lobatto nodes). v2 enforces `cfg.dg_order_qd == cfg.dg_order_dr` in the FD config validator. Under this enforcement, fault state copy at handoff is byte-equivalent — the round-trip tolerance is bit-exact for the fault, not $1e-12$ (resolves v1 R-001 / R-009).

6.3.2 Bulk velocity initialization at QD → DR (addresses v1 R-005) QD does not track bulk velocity. At handoff we set:

- $v(x) = 0$ everywhere except in a band of $\sim 3-5$ elements around the fault.

- On the fault: each side moves at $\pm \frac{1}{2} V_{\text{fault}}$ along the local strike/dip frame, where V_{fault} is the just-evaluated fault slip-rate distribution.
- Apply a Gaussian taper of width `n_taper_elements` (default 4) along the fault-normal direction to smooth the discontinuity.

The “Euler step backwards in time” idea from v1 §4.4 step 4(b) is dropped (v1 R-005 was correct: the math was meaningless). If a more accurate seed is wanted later, the right approach is a single half-step Verlet kick $v(t + dt/2) = (dt/2) \cdot \rho^{-1} \nabla \cdot \sigma$ from the projected stress; for v2 we use the simpler half-rate seed and rely on the equilibrium-correction sweep to absorb residual.

6.3.3 DR → QD verification (addresses v1 R-001 inverted-inequality) After the dynamic phase ends with $V_{\text{max}} < V_{\text{deactivate}} \approx 1e-4$ m/s, V_{final} is roughly five orders above plate rate $V_p \approx 1e-9$ m/s. So V_{final} is not a good QD seed — the QD steady-state will be near V_p , not V_{final} .

Procedure: 1. Copy (`slip1`, `slip2`, `ψ_final`) from `DOFData` into QD’s fault-state grid functions. 2. Reset QD’s time stepper to `dt_qd_default`. 3. On the next QD Mult call, the elasticity solve produces traction τ_{total} . Brent-solve $\sigma_n \cdot f(V_{\text{qd}}, \psi_{\text{final}}, a) + \eta \cdot V_{\text{qd}} = \tau_{\text{total}}$ for V_{qd} via `core/friction/rate_state.hpp::SolveBalanceForVabs`. 4. Assert $|\tau_{\text{total}} - (\sigma_n \cdot f(V_{\text{qd}}, \psi_{\text{final}}, a) + \eta \cdot V_{\text{qd}})| / |\tau_{\text{total}}| < 1e-6$. If not, abort with a diagnostic.

No need to “back-solve from V_{final} ” — Brent is robust; ψ_{final} pins the friction side; the elasticity solve pins the stress side. V_{qd} emerges from the balance.

6.4 Trigger logic and hysteresis (addresses v1 R-010)

```
// fd/trigger.hpp
namespace seas::fd {

class Trigger {
public:
    /// MPI-collective MAX over local arrays.
    static real_t AllReduceVMax(const Vector &local_slip_rates, MPI_Comm comm);
    static real_t AllReduceVMaxFromDOFData(const std::vector<seas::dr::DOFData> &d, MPI_Comm comm);

    bool ShouldEnterDynamic(real_t Vmax) const { return Vmax > V_activate_; }
    bool ShouldExitDynamic(real_t Vmax, real_t t_below) const {
        return Vmax < V_deactivate_ && t_below > exit_grace_t_;
    }
};

} // namespace seas::fd
```

Defaults (per user direction): $V_{\text{activate}} = 1e-3$ m/s, $V_{\text{deactivate}} = 1e-4$ m/s, $\text{exit_grace_t} = 0.1$ s.

The `MPI_Allreduce(MAX)` in both `AllReduceVMax*` helpers is the same pattern QD’s RK45 already uses (see `solver/time_stepper.hpp`); without it ranks would disagree on the regime and the run would deadlock.

6.5 Equilibrium-correction sweep (addresses user §8)

Confirmed needed. After QD → DR packing, we run `N_settle` (default 10) wave time steps with: - The PML profile boosted by a factor `k_settle` (default 4) — absorbs the projection-induced wave packets that originate from the bulk-velocity seed (§6.3.2). - The friction solver still active — slip continues to evolve naturally. - Output suppressed (these settling steps don’t pollute station I/O).

After settling completes, the PML profile is reset to its production value and normal time loop continues.

Why we need it: the bulk-velocity seed is necessarily approximate (zero in the bulk, half-rate near the fault, Gaussian taper). The mismatch with the “true” (unknown) initial velocity field for the dynamic phase generates spurious P/S waves emanating from the fault region. Without absorption these contaminate rupture observation in the early dynamic phase.

Why not skip it: dynamic_rupture_plan v4 §5.2 simulations show ~10% peak slip-rate error from skipped settling on a TPV102-like cold start.

TOML knob: [hybrid] N_settle = 10, k_settle = 4.0 — overridable per run; v1 R-006 noted “could wash out a real rupture” — the cycle case differs from cold-start nucleation because the rupture is being initiated by QD-evolved slip-rate, not by a nucleation pulse. Settling absorbs spurious waves but does not affect the slow QD-induced quasistatic evolution near the fault (no wave content there to absorb).

6.6 Mesh strategy (addresses user §9)

Two meshes: - Verification mesh bp5/mesh/hybrid_verification_3km.{geo,msh} — small box (~30×30×30 km), tagged with §3.3 contract (FAULT=3, NATURAL=1, DIRICHLET=5, PML=7), DG order matched to BP5. Used for seas_fd workflow tests, all phases, locally on np ≤ 10 (per saved feedback). - BP5 production mesh bp5/mesh/bp5_tandem_exact.msh — once we add a PML outer layer (one-time mesh regeneration), this is the production cycle-mode mesh. Frontera-only.

Both meshes use the same TOML schema; only the file path differs.

6.7 What CAN be PML+loading combined? (addresses user §7)

Concretely: - PML face attribute (tag 7): outer-of-outer, never carries Dirichlet. - Dirichlet face attribute (tag 5): vertical sides not at the absorbing boundary; carry plate loading $u_X = \pm \text{sgn}(Y) \cdot V_p \cdot t / 2$. - These tag sets are disjoint in the new BP5 cycle mesh; co-existence is geometric, not algorithmic.

During QD: PML is a no-op (no waves to absorb; the static elasticity solve does not see PML damping). Dirichlet loading is active.

During DR: PML actively damps. Dirichlet loading is also active — each DR time step’s right-hand side includes the small $V_p \cdot dt/2$ increment to far-field displacement. For TPV102-style runs without loading we use $V_p = 0$. For BP5 cycle mode $V_p = 1e-9$ m/s is on continuously (no toggling at handoff).

7. TOML configuration (per user §11)

7.1 Three schemas, one shared base

```
config/schemas/
├─ seas_common.toml      # shared sub-schema: mesh, friction, output, mpi
├─ seas_qd.toml          # QD-only: extends common with [qd] block
├─ seas_dr.toml          # DR-only: extends common with [dr] block (benchmark, nucleation, pml)
└─ seas_fd.toml          # FD: extends common with [hybrid] + [qd] + [dr]
```

7.2 Common base (seas_common.toml example)

```
[mpi]
np = 8

[mesh]
file = "bp5/mesh/bp5_tandem_exact.msh"
fault_tag    = 3  # SEASBoundaryTags::FAULT
natural_tag   = 1
```

```

dirichlet_tag = 5
pml_tag       = 7
dg_order      = 4  # MUST match across QD and DR for seas_fd (§6.3.1)

[material]
rho  = 2670.0      # kg/m^3
lambda = 32.04e9   # Pa
mu   = 32.04e9

[friction]
law = "rate_state_aging"      # only choice for seas_qd / seas_fd; seas_dr can also pick "rate_state_srw"
f0  = 0.6
V0  = 1e-6
V_init = 1e-9                # plate-rate scale

[friction.fields]
# Spatial profiles given as TOML expressions; loader parses to std::function.
a  = "0.004 + 0.0 * z"       # uniform a = 0.004 (TPV102) or BP5 piecewise
b  = "0.014"
Dc = "0.14 + 0.0 * z"

[output]
paraview_dir = "results/run_XXX"
station_freq_yr = 0.5        # output cadence in QD phase
station_freq_dyn = 1.0e-3    # cadence during dynamic phase (s)

```

7.3 FD-specific block

```

# seas_fd.toml inherits seas_common; adds:
[hybrid]
V_activate   = 1.0e-3        # m/s, QD → DR threshold
V_deactivate = 1.0e-4        # m/s, DR → QD threshold
exit_grace_t = 0.1           # seconds V < V_deactivate before regime switch
N_settle     = 10            # wave steps with boosted PML at handoff
k_settle     = 4.0
n_taper_elements = 4         # bulk velocity seed taper width

[qd]
dt_initial_yr = 0.001
dt_max_yr     = 0.1

[dr]
ader_order = 4
cfl         = 0.5
pml_thickness_km = 8.0
pml_d_max    = 5.0e7

```

7.4 CLI shim

For one release, every existing CLI flag (e.g., `--v-nuc 0.03`) translates to a TOML override `[friction]` `V_nuc = 0.03`. After that, CLI is removed. This keeps existing test scripts working through Phase 0–2 of the implementation.

7.5 Validation

The config loader runs a validation pass: - All required keys present. - seas_fd: dg_order consistent (per §6.3.1); $V_{\text{activate}} > V_{\text{deactivate}}$; $\text{exit_grace_t} \geq 0$. - seas_qd: $\text{friction.law} \in \{\text{rate_state_aging}\}$. - seas_fd: $\text{friction.law} == \text{rate_state_aging}$ (per user §5: only this law in cycle mode). - seas_dr: $\text{friction.law} \in \{\text{rate_state_aging}, \text{rate_state_srw}, \text{lsr}\}$.

Library: toml++ (header-only, BSD-3, already common in research codes; bundled in external/toml/). One new dependency.

8. Workflow per phase (high-level)

The full phase plan is in the appendix (§13). High-level summary:

Phase	What lands	Apps changed	Risk
0. Foundations	TOML loader, core/, sign-convention freeze, mesh-tag table (no coord fallback). All current goldens still pass.	qd, dr	Low. No physics change; pure plumbing.
1. seas_qd standalone	drivers/seas_qd_driver.cpp, qd::Application. BP5 v62 reproduces.	qd	Low. Repackages existing code.
2. seas_dr standalone	drivers/seas_dr_driver.cpp + framework strategy templates. TPV102/104/205 reproduce.	dr	Medium. Touches the per-benchmark trio.
3. seas_fd skeleton	HybridOperator, StateBridge, Trigger. Synthetic-trigger smoke test passes.	fd	Medium. New code, shape uncertainty.
4. seas_fd handoffs	QD→DR and DR→QD bridge methods, equilibrium correction. Round-trip with no physical evolution preserves fault state bit-exact.	fd	High. Where the physics is.
5. seas_fd cycle smoke	100-yr smoke on verification mesh; one synthetic event detected.	fd	Medium. Integration.
6. seas_fd BP5 cycle	First-event reproduction on BP5 mesh, Frontera.	fd	High. Production.

Each phase is independently mergeable — the code compiles, the previously-passing tests still pass, and the new code is gated behind config to avoid breaking unrelated runs.

9. Acceptance criteria (top-level)

1. seas_qd reproduces BP5 v62 golden (job7639398) byte-for-byte.

2. seas_dr reproduces TPV102, TPV104, TPV205 goldens to byte-for-byte (or to documented numerical tolerance $\leq 1e-12$ if Phase 0's TOML round-trip introduces measurable noise).
3. seas_fd runs a 1-cycle synthetic-trigger smoke on the verification mesh without NaN, with regime switches in both directions, finishing within $5\times$ the QD-only wall-clock time on the same mesh.
4. seas_fd reproduces the first earthquake recurrence (cycle 0 $\rightarrow$ cycle 1) of BP5 on the production mesh within published reference tolerance (Kaneko et al. 2008 / SCEC BP5).
5. No coordinate-based boundary detection anywhere in core/mesh/. Verified by grep; CI fails if coord_tol-style fallbacks reappear.
6. No CG $\leftrightarrow$ DG projection anywhere in fd/. Verified by grep.
7. All three apps build independently (separate CMakeLists.txt targets); deleting any of qd/, dr/, fd/ libraries does not break the others' builds.
8. Single law in QD/FD — friction TOML loader rejects lsw and rate_state_srw for seas_qd and seas_fd configs.
9. Round-trip QD $\rightarrow$ DR (with no rupture) $\rightarrow$ QD on the verification mesh: fault-DOF state preserved bit-exact (because §6.3.1 guarantees the fault copy is identity); bulk u recovered to $\leq 1e-8 / |u|$ (DG L²-projection error after equilibrium-correction sweep).

10. The implementation appendix begins

Sections §11–§13 give file-level signatures, unit tests, and the full phase-by-phase build order. Read them when you are ready to implement; the body above is the contract.

11. File and module layout (full)

```
miniapps/seas/
├─ core/                                # libseas_core
│  ├─ conventions.hpp                  # sign / threshold constants
│  ├─ friction/
│  │  ├─ rate_state.hpp                # FrictionCoefficient, SolveBalanceForVabs, AdvancePsiAging
│  │  └─ rate_state_params.hpp
│  ├─ fault/
│  │  └─ fault_basis.hpp              # promoted from miniapps/seas/fault/fault_basis.hpp
│  ├─ mesh/
│  │  ├─ boundary_tags.hpp            # SharedFaceTagTable; NO coordinate fallback
│  │  └─ par_mesh_loader.hpp          # LoadParMesh + tag validation
│  ├─ config/
│  │  ├─ toml_loader.hpp
│  │  └─ schemas/
│  │     ├─ seas_common.cpp           # impl
│  │     ├─ seas_qd.cpp
│  │     ├─ seas_dr.cpp
│  │     └─ seas_fd.cpp
│  └─ cli_shim.hpp                    # legacy --foo  $\rightarrow$  TOML translation (Phase 0–2 only)
├─ io/
│  ├─ paraview_writer.hpp
│  └─ station_writer.hpp
├─ qd/                                # libseas_qd
│  ├─ application.hpp                 # qd::Application
│  ├─ elasticity_operator.hpp          # was domain/elasticity_operator.hpp; routed through core
│  └─ rate_state_fault.hpp            # was fault/rate_state_fault.hpp; routed through core
```

```

└─ time_stepper.hpp          # was solver/time_stepper.hpp; unchanged
└─ ...

└─ dr/                      # libseas_dr
    └─ wave_operator.{hpp,cpp,inl}
    └─ fault_face_flux.{hpp,cpp}
    └─ godunov_flux.{hpp,cpp}      # BuildFrame deleted; calls core/fault/FaultBasis
    └─ pml_layer.{hpp,cpp}
    └─ framework/
        └─ runtime_loop.hpp        # DRRuntimeLoop<...>
        └─ substep_iterator.hpp
        └─ friction_strategy.hpp
        └─ state_evolution_strategy.hpp
        └─ nucleation_strategy.hpp
    └─ benchmarks/
        └─ tpv102.{hpp,cpp}        # MakeTPV102Config, registered with framework
        └─ tpv104.{hpp,cpp}
        └─ tpv205.{hpp,cpp}

└─ fd/                      # libseas_fd
    └─ hybrid_operator.{hpp,cpp}
    └─ state_bridge.{hpp,cpp}
    └─ trigger.{hpp,cpp}
    └─ equilibrium_correction.{hpp,cpp} # N_settle wave-step sweep

└─ drivers/
    └─ seas_qd_driver.cpp
    └─ seas_dr_driver.cpp
    └─ seas_fd_driver.cpp

└─ tests/
    └─ unit/
        └─ test_friction_rate_state.cpp
        └─ test_fault_basis.cpp
        └─ test_boundary_tag_table.cpp
        └─ test_toml_schemas.cpp
        └─ test_state_bridge.cpp
        └─ test_trigger.cpp
        └─ test_equilibrium_correction.cpp
    └─ integration/
        └─ test_seas_qd_bp5_smoke.cpp
        └─ test_seas_dr_tpv102_smoke.cpp
        └─ test_seas_fd_synthetic_trigger.cpp
        └─ test_seas_fd_round_trip.cpp

```

The existing folder structure (solver/, fault/, friction/, domain/, dynamic/, bp5/) is preserved during Phases 0–2 (each old file becomes a one-line forwarding header to its new location). Phase Z (after all goldens reproduce) deletes the forwarding headers.

12. Function-level signatures and unit tests (the long list)

For each major surface introduced above, the corresponding signature and at least one named test case. Implement in the order they are listed; each comes with a sketch of the test that gates it.

12.1 core/friction/rate_state.hpp

```
real_t FrictionCoefficient(real_t V_abs, real_t psi, real_t a, real_t V0);
real_t SolveBalanceForVabs(real_t tau_total_abs, real_t sigma_n, real_t psi,
    real_t a, real_t V0, real_t eta_s,
    real_t tol = 1e-12, int max_iter = 200);
real_t AdvancePsiAging(real_t psi_n, real_t V_abs, real_t dt,
    real_t a, real_t b, real_t Dc, real_t f0, real_t V0);
```

Tests (tests/unit/test_friction_rate_state.cpp): - TestFrictionCoefficient_AsinhMatchesRegularizedFormula — pin $f(V=V_0, \psi=a, a=0.01, V_0=1e-6)$ to $0.01 \cdot \text{asinh}(0.5 \cdot e^1)$ to $1e-15$. - TestSolveBalanceForVabs_SteadyState — given $f(V_{ss}, \psi_{ss}, a)$ at slip-rate V_p , recovered V from balance is within $1e-12$ of V_p . - TestSolveBalanceForVabs_FrictionlessLimit — when $\psi \ll -1$, returns τ/η without converging. - TestAdvancePsiAging_ZeroV — at $V=0$, ψ relaxes monotonically toward $f_0 + b \cdot \ln(V_0/V_{init})$. - TestAdvancePsiAging_SteadyState — at $V=V_{ss}$ given by the steady balance, $d\psi/dt = 0$ to $1e-14$.

12.2 core/fault/fault_basis.hpp

```
class FaultBasis { /* see §3.2 */ };
```

Tests (tests/unit/test_fault_basis.cpp): - TestFaultBasis_VerticalY0_Tangents — for $n = (0, -1, 0)$, up $= (0, 0, 1)$: $t_1 = (0, 0, -1)$, $t_2 = (1, 0, 0)$. - TestFaultBasis_RotateRoundTrip — $\text{RotateFaultToWorld}(\text{RotateWorldToFault}(v)) == v$ to $1e-15$. - TestFaultBasis_StressRotation — diagonal world stress $\text{diag}(\sigma_{xx}, \sigma_{yy}, \sigma_{zz}) \rightarrow$ fault-frame value computed analytically. - TestFaultBasis_DipDownward — $t_1 \cdot \hat{z} < 0$ (the dip vector points down).

12.3 core/mesh/boundary_tags.hpp

```
struct BoundaryTags { /* §3.3 */ };
class SharedFaceTagTable { /* §3.3 */ };
```

Tests (tests/unit/test_boundary_tag_table.cpp): - TestSharedFaceTagTable_LocalFaultMatchesBdrAttribute — interior fault face on a 1-rank mesh has tag 3. - TestSharedFaceTagTable_SharedFaultPropagatesAcrossPartition — $np=4$ split mesh: shared fault face has tag 3 on both ranks (no coord fallback used). - TestSharedFaceTagTable_NonFaultSha — verifies false-positive immunity. - TestSharedFaceTagTable_GrepNoCoordFallback — meta-test: `grep -r 'coord_tol' core/mesh/` returns 0 hits.

12.4 core/config/toml_loader.hpp

```
struct CommonConfig { /* mesh, material, friction, output */ };
struct QDConfig      : CommonConfig { /* [qd] block */ };
struct DRConfig      : CommonConfig { /* [dr] block, benchmark name */ };
struct FDConfig      : CommonConfig { /* [hybrid], [qd], [dr] */ };
```

```
QDConfig LoadQDConfig(int argc, char* argv[]);
DRConfig LoadDRConfig(int argc, char* argv[]);
FDConfig LoadFDConfig(int argc, char* argv[]);
```

Tests (tests/unit/test_toml_schemas.cpp): - TestQDLoad_RejectsLSW — config with `friction.law = "lsw"` causes loader to abort. - TestFDLoad_RejectsRateStateSRW — same for SRW in FD config. - TestFDLoad_RequiresMatchingDGOrder — config with different qd/dr DG orders aborts. - TestFDLoad_VActivateMustExceedVDeactivate — invalid threshold ordering aborts. - TestQDLoad_DefaultsMatchExistingBP — loaded QDConfig from `bp5_default.toml` matches today's BP5 driver defaults field-for-field.

12.5 fd/state_bridge.hpp

```
class StateBridge { /* §6.3 */ };
```

Tests (tests/unit/test_state_bridge.cpp): - TestQDtoDR_FaultStateBitExact — round-trip fault DOF copy preserves all 3 fields per DOF to bit-exact (no projection, identity copy). - TestQDtoDR_BulkStressFromGradU — synthetic linear $u(x) \rightarrow \sigma$ matches analytic $C:\nabla u$ to $1e-13$. - TestDRtoQD_BalanceVerification_PassesAtSteadyState — synthetic DR end-state at steady-rate gives equilibrium error $< 1e-7$. - TestDRtoQD_BalanceVerification_FailsOnInconsis — perturb ψ by 10% $\rightarrow$ equilibrium check fires. - TestQDtoDR_BulkVelocitySeed_TaperDecaysToZero — at distance $\geq n_taper_elements$, $|v| < 1e-3 \cdot \max(|v|)$. - TestQDtoDR_NoTau00verwrite — confirms `data.tau1_0` and `data.tau2_0` retain their static config values across handoff (resolves v1 R-002). - TestQDtoDR_NoTauNucWrite — confirms `tau*_nuc` slots are 0 after handoff (no nucleation in cycle mode).

12.6 fd/trigger.hpp

```
class Trigger { /* §6.4 */ };
```

Tests (tests/unit/test_trigger.cpp): - TestAllReduceVMax_AgreesAcrossRanks — $np=4$: per-rank V_max differs, allreduced value is the global MAX to $1e-15$. - TestEnterDynamic_AtThreshold — $V_max = 1e-3 + \epsilon$ returns true. - TestExitDynamic_RequiresGracePeriod — $V_max < V_deactivate$ but $t_below < grace \rightarrow$ still false. - TestHysteresisPreventsToggle — synthetic V_max trajectory crossing both thresholds rapidly: regime switches at most once per direction.

12.7 fd/equilibrium_correction.hpp

```
class EquilibriumCorrection {
public:
    /// Run N_settle wave steps with PML profile boosted by k_settle.
    /// The wave op's PML coefficient is multiplied by k_settle for the duration
    /// of these steps and reset afterwards.
    void Run(seas::dr::WaveOperator &wave, std::vector<seas::dr::DOFData> &dof,
            int N_settle, real_t k_settle, real_t &t, real_t &dt);
};
```

Tests (tests/unit/test_equilibrium_correction.cpp): - TestSettling_ReducesBulkKineticEnergy — synthetic post-handoff state with pure bulk wave content: KE after `N_settle` $< 0.1 \times$ KE before. - TestSettling_PreservesFaultSlip — slip values change by $< 1e-6$ m over 10 settling steps (no rupture-driving stress in test). - TestSettling_ResetsPMLAfter — PML profile after settling matches profile before (no leak).

12.8 fd/hybrid_operator.hpp

```
class HybridOperator { /* §6.2 */ };
```

Integration tests (tests/integration/test_seas_fd_synthetic_trigger.cpp): - TestSyntheticTrigger_QDtoDRtoQD — verification mesh, force $V_max > V_activate$ artificially, run 100 macro steps, confirm one regime switch in each direction, no NaN, fault state values finite. - TestRoundTrip_NoEvolution — start with a frozen QD state at steady rate, force-trigger to DR, force-trigger back to QD, verify final fault DOF state == initial (modulo equilibrium-correction-induced slip $\leq 1e-6$ m).

13. Phase plan (the one-page version of §8 expanded)

13.1 Phase 0 — Foundations (1 week)

Goal: TOML, core/, mesh-tag table land. All existing tests pass.

Files added: - core/conventions.hpp, core/config/toml_loader.hpp + schemas, core/friction/rate_state.hpp, core/fault/fault_basis.hpp, core/mesh/boundary_tags.hpp. - tests/unit/test_friction_rate_state.cpp, test_fault_basis.cpp, test_boundary_tag_table.cpp, test_toml_schemas.cpp.

Files modified: existing friction/dieterich_ruina.hpp, friction/state_evolution.hpp, fault/fault_basis.hpp, domain/seas_boundary_tags.hpp become forwarding shims.

Acceptance: BP5 v62 golden, TPV102/104/205 goldens reproduce byte-for-byte. All new unit tests pass.

13.2 Phase 1 — seas_qd standalone (1 week)

Goal: drivers/seas_qd_driver.cpp runs BP5 from a TOML config.

Files added: qd/application.hpp, drivers/seas_qd_driver.cpp, tests/integration/test_seas_qd_bp5_smoke.cpp.

Acceptance: BP5 v62 reproduces from seas_qd --config bp5.toml. CLI shim covers all today's seas_driver flags.

13.3 Phase 2 — seas_dr standalone (2 weeks)

Goal: drivers/seas_dr_driver.cpp runs all three TPV benchmarks from TOML; framework strategy templates land.

Files added: dr/framework/{runtime_loop, substep_iterator, friction_strategy, state_evolution_strategy, nucleation_strategy}.hpp, dr/benchmarks/{tpv102, tpv104, tpv205}.hpp, drivers/seas_dr_driver.cpp, integration tests.

Files modified: existing dynamic/tpv1XX_* files become forwarding shims (deleted in Phase Z).

Acceptance: TPV102/104/205 goldens reproduce. seas_dr --config tpv102.toml works on local np ≤ 10 (per saved feedback) for tpv102/tpv104; TPV205 may need Frontera np > 10 for golden-quality LSW (per tpv205_lsw_native_fields_plan Phase 4).

13.4 Phase 3–6 — seas_fd (4–5 weeks)

Phases as in §8. Phase 6 requires Frontera approval (per saved feedback feedback_frontera_approval).

13.5 Phase Z — Cleanup (0.5 week)

After all goldens reproduce on the new code paths, delete forwarding shims, retire the legacy folders (dynamic/tpv1XX_*, domain/seas_boundary_tags.hpp, etc.).

13.6 v1 review-finding traceability

v1 finding	Resolution in v2	Section
R-001 V_deactivate inverted inequality	Defaults set per user direction (V_act=1e-3, V_deact=1e-4); §6.3.3 verification uses Brent back-out, not direct V_final.	§3.4, §6.3.3
R-002 tau_0 overwrite	StateBridge does NOT touch tau*_0 or tau*_nuc at handoff; verified by TestQDtoDR_NoTau00overwrite	§6.3, §12.5
R-003 TimeDependentOperator base contradiction	HybridOperator does NOT inherit TimeDependentOperator; owns its own time loop	§6.2

v1 finding	Resolution in v2	Section
R-004 solver/ no-edit conflict	“do not edit” zone dropped per user §1; structural changes authorized	§1.1
R-005 Euler backwards in time	Dropped; replaced with half-rate seed + equilibrium-correction sweep	§6.3.2, §6.5
R-006 V from latest RHS	Bridge re-solves Brent at handoff via core/friction::SolveBalanceForVabs	§6.3.3
R-007 nucleation overwrite/accumulate contract	NucleationSlot enum + per-tag static constexpr slot	§5.3
R-008 CG-DG order assumption	Both phases are DG; no CG anywhere; FD config validator enforces matched DG order	§1.2, §6.3.1, §7.5
R-009 1e-12 vs 1e-6 round-trip	Bit-exact for fault state (identity copy); §9 acceptance #9 specifies bulk tolerance separately	§9
R-010 dyn_stepper interface	dyn_wave_.Step(dof_data, t, dt, iter_) advances both bulk Q and DOFData	§6.2
R-011 B.0 mesh acceptance ambiguity	Two meshes per §6.6 (verification + production); each has its own config	§6.6
R-012 V_w in DOFData	LSW excluded from QD/FD per user §5; SRW (V_w) only in seas_dr where DOFData append is allowed	§1.2, §3.5

14. Open questions for v3

These were not in v1 and are new in v2:

1. CLI shim sunset date. When do we delete the --foo-style legacy flags? Default proposal: at end of Phase 2.
2. Verification mesh sharing. Should seas_qd and seas_dr also support running on the verification mesh for cross-app sanity checks? Default: yes, with their own configs.
3. Output co-location. When seas_fd switches regimes mid-run, do we want one continuous time-series file or two (one per regime)? Default: one, with a regime column.
4. PML verification benchmark. What’s the canonical wide-angle / surface-wave incidence test? SCEC PEER-VS30? Defer until Phase 2.

End of v2 draft. The architecture sketch in §1–§9 is the load-bearing contract. §10–§13 are the build instructions. Open questions in §14 to be resolved before Phase 3.